

Project Sakura 第1卷 基礎編 v0.1

第1章 + 第2章 完全版

Project Sakura 第1巻 基礎編 v0.1

第1章 Project Sakuraとは

AIを活用した資産ブログと運営システムを構築するための設計書兼教材です。

1.1 Project Sakuraとは

Project

Sakuraは、AIを活用しながら資産ブログを構築し、その運営作業も少しずつ仕組み化していくプロジェクトです。

単にブログ記事を書くことだけを目的とするではありません。

企画、調査、記事作成、画像作成、公開、アクセス確認、改善、データ蓄積という一連の流れを整理し、継続的に運営できる仕組みを作ることを目指します。

1.2 Project Sakuraの目的

Project Sakuraの目的は、長期的に育てていける資産ブログと、その運営を支援するシステムを構築することです。

最初から大きなシステムを完成させるのではなく、小さな機能を一つずつ作り、実際に動かしながら改善していきます。

1.3 AIを活用する

Project Sakuraでは、AIを作業を支援するための重要な道具として利用します。

AIには、アイデア出し、情報整理、文章構成、下書き作成、表現改善、プログラム作成支援、エラー原因の整理などを担当させます。

ただし、AIにすべてを任せることが目的ではありません。

AIが提案する

↓

人が確認する

↓

必要なら修正する

↓

採用する

という流れを基本とします。

1.4 Pythonを学ぶ

Project Sakuraでは、Pythonを使って少しずつ自動化を進めます。

最初は、

```
print("こんにちは！Project Sakuraへようこそ！")
```

という非常に小さなプログラムから始めます。

このプログラムは、画面に文字を表示するだけの非常に小さなプログラムです。

しかし、プログラミングでは、このような小さな処理を組み合わせることで大きな仕組みを作ることができます。

Project Sakuraでは、

文字を表示する

↓

ファイルを扱う

↓

データを扱う

↓

処理を自動化する

↓

Webサービスと連携する

↓

ブログ運営を支援する

というように、少しずつ発展させていきます。

プログラミング未経験者でも理解できるように、実際に動かしながら学ぶことを基本とします。

1.5 GitHubは何のために使うのか

Project SakuraではGitHubを利用して、プロジェクトのファイルを管理します。

GitHubには、ファイルを保存するだけでなく、変更履歴を残せるという大きな特徴があります。

例えば、

Version 0.1

↓

Version 0.2

↓

Version 0.3

というように、プロジェクトがどのように変化したのかを記録できます。

もし途中でファイルを変更して問題が発生した場合でも、以前の状態を確認したり、変更内容を調べたりできます。

今回作成した、

`project-sakura`

というリポジトリも、Project Sakuraの開発記録を管理するために利用します。

GitHub Desktopを利用することで、Gitのコマンドを最初から覚えなくても、画面上の操作でファイルの変更を確認し、保存できます。

1.6 現在のProject Sakura

本書の作成開始時点で、基本的な開発環境の準備は完了しています。

現在使用する主な環境は次のとおりです。

項目 状態

GitHubアカウント 作成完了

GitHub Desktop インストール完了

VS Code インストール完了

Python インストール完了

`project-sakura` 作成済み

GitHubへの保存 確認済み

Pythonプログラムの実行 確認済み

第1章原稿 作成中

Project Sakuraの基本的なフォルダ構成は、次のように整理していきます。

`project-sakura`

```
|
|
|— docs
| |
| |— CHANGELOG.md
| |
| |— Vol01_基礎編
| |
| |— images
| |
| |— source
```

```
| | └─ Chapter01.md
| |
| └─ README.md
|
└─ python
  | └─ lesson01.py
  |
  └─ .gitattributes
  |
  └─ README.md
```

1.7 本書の進め方

本書では、最初から高度な知識を必要としません。

基本的には、

説明

↓

実際に操作

↓

結果を確認

↓

問題があれば修正

↓

次へ進む

という流れで進めます。

分からない用語が出てきた場合は、その都度説明します。

操作についても、

「どこをクリックするのか」

「何を入力するのか」

「何が表示されれば成功なのか」

まで、できるだけ具体的に説明します。

本書は、プログラマーだけを対象とした専門書ではありません。

プログラミング未経験者でも、Project Sakuraを少しずつ作れることを目標とします。

1.8 Project Sakuraの最終目標

Project Sakuraの最終目標は、単にブログを1つ作成することではありません。

目指しているのは、

企画

↓

調査

↓

AIによる支援

↓

記事作成

↓

画像作成

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の記事へ

という一連の流れを、継続的に回せる仕組みです。

さらに、その仕組み自体も改善していきます。

最初は手作業だった作業を、

手作業

↓

手順化

↓

半自動化

↓

自動化

へと発展させていきます。

これがProject Sakuraの大きなテーマです。

1.9 このプロジェクトで大切にすること

Project Sakuraでは、次の5つを大切にします。

1. 小さく始める

最初から完璧なシステムを作ろうとしません。

小さな機能を一つずつ作ります。

2. 実際に動かして確認する

説明を読むだけではなく、実際に操作して結果を確認します。

3. 失敗も記録する

エラーや失敗も重要な情報です。

同じ問題を繰り返さないために、原因や解決方法を記録します。

4. 作ったものを残す

記事、コード、設計、手順などをGitHubに保存します。

5. 継続できる仕組みにする

一時的に大量の作業をするのではなく、無理なく長期間続けられる仕組みを目指します。

1.10 初心者から始めるということ

Project Sakuraでは、最初から専門知識を持っている必要はありません。

分からないことがあれば、その都度調べます。

エラーが出たら、その原因を確認します。

操作を間違えたら、戻してやり直します。

重要なのは、

「分からないから進めない」

ではなく、

「分からないところを一つずつ解決する」

という進め方です。

プログラムもブログも、一度に完成するものではありません。

小さな完成を積み重ねることで、最終的に大きなシステムになります。

1.11 現在までに行ったこと

Project Sakuraを開始するために、これまでに次の環境を準備しました。

GitHub

GitHubアカウントを作成しました。

Googleアカウントを利用して登録を行い、Project Sakuraの保存場所となるGitHub環境を準備しました。

GitHub Desktop

WindowsでGitHubを扱いやすくするため、GitHub Desktopをインストールしました。

GitHub Desktopを利用することで、コマンドを大量に入力しなくても、

変更されたファイルを確認する

変更内容を確認する

Commitする

GitHubへ送信する

過去の変更履歴を見る

といった操作ができます。

VS Code

プログラムや文章ファイルを編集するため、Visual Studio Codeをインストールしました。

Project Sakuraでは、Pythonプログラムだけではなく、Markdown形式の原稿もVS Codeで編集します。

Python

Pythonをインストールしました。

そして最初のプログラムとして、

```
print("こんにちは！ Project Sakuraへようこそ！")
```


を作成し、実際に実行できることを確認しました。

これは小さな一歩ですが、Project Sakuraで初めて動かしたプログラムです。

1.12 ドキュメント管理

Project Sakuraでは、ドキュメントを整理して保存します。

現在は、

`project-sakura/docs/`

をドキュメント管理用の場所として使用しています。

第1巻の基礎編については、

`project-sakura/docs/Vol01_基礎編/`

の中に原稿を保存します。

さらに、

`source`

フォルダには、各章のMarkdown原稿を保存します。

例えば第1章は、

`Chapter01.md`

として管理します。

このMarkdown原稿を元にして、最終的にはWord版やPDF版などの正式なドキュメントへ展開していきます。

1.13 原稿と正式版の関係

Project Sakuraでは、チャットの内容をそのまま本にするではありません。

まず原稿を作ります。

その原稿を確認します。

必要な修正を行います。

内容が確定した段階で正式版として保存します。

基本的な流れは、

原稿作成

↓

内容確認

↓

修正

↓

章の完成

↓

正式版へ反映

↓

GitHubへ保存

とします。

これにより、Project Sakuraの本そのものも、バージョン管理された成果物として残すことができます。

1.14 Project Sakuraは「本」と「システム」を同時に作る

Project Sakuraには、2つの側面があります。

1つ目は、ブログを運営するためのシステムを作ることです。

2つ目は、そのシステムを作る過程を本として記録することです。

つまり、

ブログ

+

AI

+

Python

+

GitHub

+

WordPress

+

運営ノウハウ

を一つのプロジェクトとしてまとめます。

そのため、本書は単なる操作マニュアルではありません。

Project

Sakuraが成長していく過程そのものを記録する「開発記録」であり、「教材」であり、「設計書」でもあります。

1.15 第1章まとめ

Project Sakuraは、AIを活用しながら資産ブログを構築・運営し、その運営作業も少しずつ仕組み化していくプロジェクトです。

本プロジェクトでは、

AIを活用する

Pythonで自動化する

GitHubで管理する

VS Codeで編集する

WordPressで公開する

データを蓄積する

改善を繰り返す

という流れを作っていきます。

そして、作成したものだけではなく、作業方法や失敗、改善方法についても記録していきます。

Project Sakuraは、

「ブログを作るプロジェクト」

であると同時に、

「ブログを育てる仕組みを作るプロジェクト」

でもあります。

最初から大きなシステムを完成させる必要はありません。

小さなプログラムを動かす。

小さな記事を作る。

小さな改善を行う。

その一つ一つを保存する。

そして、それらを積み重ねていきます。

これがProject Sakuraの基本的な進め方です。

Chapter Check

第1章を読み終えたら、次の質問を確認してみましょう。

Q1

Project Sakuraの目的は何ですか？

Q2

AIはProject Sakuraでどのような役割を担当しますか？

Q3

Pythonは何のために利用しますか？

Q4

GitHubを利用する理由は何ですか？

Q5

Project Sakuraが目指している運営の循環とは何ですか？

Q6

なぜ作った記事やプログラムだけでなく、作業手順や失敗も記録するのでしょうか？

第2章への予告

第1章では、Project Sakuraが何を目標しているのかを確認しました。

第2章からは、いよいよ「何を作るのか」を具体的に設計していきます。

ブログ、AI、Python、GitHub、WordPress、データ管理などが、それぞれどのようにつながるのかを整理します。

そして、Project Sakura全体の構造を明確にします。

第2章では、

「Project Sakura全体設計」

をテーマとして進めます。

Project Sakura 第1巻 基礎編 v0.1

第2章 Project Sakura全体設計

2.1 はじめに

第1章では、Project Sakuraが何を目指しているのかを確認しました。

Project

Sakuraは、AIを活用しながら資産ブログを構築し、その運営作業も少しずつ仕組み化していくプロジェクトです。

第2章では、その目的を実現するために、Project

Sakura全体をどのような仕組みにしていくのかを整理します。

ここでは、いきなり細かいプログラムを作るのではなく、まず全体像を把握します。

大きな仕組みを作るときには、最初に「何を作るのか」「それぞれの部品がどのようにつながるのか」を決めておくことが重要です。

Project Sakuraでは、

企画 ↓ 調査 ↓ AIによる支援 ↓ 記事作成 ↓ 画像作成 ↓ 公開 ↓ アクセス確認 ↓

改善 ↓ データ蓄積

という流れを基本とします。

そして、この流れを繰り返しながら、手作業だった部分を少しずつ仕組み化していきます。

2.2 Project Sakura全体の考え方

Project Sakuraを一言で表すなら、

「ブログ運営を継続できる仕組みを作るプロジェクト」

です。

単純にブログを作るだけなら、WordPressを用意して記事を書けば始めることができます。

しかし、長期間ブログを運営する場合には、それだけでは十分ではありません。

記事のテーマを考える必要があります。

情報を調査する必要があります。

記事を作成する必要があります。

画像を準備する必要があります。

公開後のアクセス状況を確認する必要があります。

さらに、結果を見ながら次の記事を改善する必要があります。

Project Sakuraでは、これらの作業を一つずつ整理します。

そして、

「人が判断する部分」

と

「コンピューターに任せられる部分」

を分けて考えます。

最終的には、人が重要な判断を行い、コンピューターやAIが繰り返し作業を支援する形を目指します。

2.3 Project Sakuraを構成する主要要素

Project Sakuraでは、主に次の要素を利用します。

- AI
- Python
- GitHub
- GitHub Desktop
- VS Code
- WordPress
- データ管理
- 画像作成
- アクセス解析

これらは、それぞれ別々のものとして存在するものではありません。

役割を分担しながら、一つの運営システムとしてつなげていきます。

例えば、

AI ↓ 企画・調査・文章作成を支援

Python ↓ 繰り返し作業やデータ処理を自動化

GitHub ↓ プログラムやドキュメントを管理

VS Code ↓ プログラムや原稿を編集

WordPress ↓ 記事をWeb上へ公開

アクセス解析 ↓ 公開後の結果を確認

という関係になります。

2.4 AIの役割

Project Sakuraでは、AIを重要な支援役として利用します。

ただし、AIにすべてを任せることを目的にはしません。

AIには得意な作業があります。

例えば、

- アイデアを出す
- 情報を整理する
- 文章の構成を考える
- 下書きを作る
- 表現を改善する
- プログラム作成を支援する
- エラーの原因を調べる
- 作業手順を整理する

といった作業です。

一方で、最終的な判断が必要な部分もあります。

例えば、

- その情報を本当に公開してよいか
- 内容が正しいか
- 読者にとって役立つか
- Project Sakuraの方針に合っているか

- 実際のサイトで公開するか

といった判断です。

そのため、Project Sakuraでは、

AIが提案する ↓ 人が確認する ↓ 必要なら修正する ↓ 採用する

という流れを基本とします。

AIは「全部を任せる存在」ではなく、「作業を前に進めるための支援役」として利用します。

2.5 Pythonの役割

Pythonは、Project Sakuraの自動化を担当する重要な要素です。

最初は非常に簡単なプログラムから始めます。

例えば、

```
``` python
print("こんにちは！Project Sakuraへようこそ！")
```
```

というプログラムです。

このプログラムは画面に文字を表示するだけです。

しかし、Pythonでは、このような小さな処理を組み合わせ、より大きな処理を作ることができます。

Project Sakuraでは、今後、

文字を表示する ↓ ファイルを読む ↓ ファイルへ保存する ↓ データを整理する
↓ 繰り返し処理する ↓ Webサービスと通信する ↓ ブログ運営を支援する

という順番で、少しずつPythonの利用範囲を広げていきます。

最初から難しい自動化を作るのではなく、動作を確認しながら段階的に発展させます。

2.6 GitHubの役割

GitHubは、Project Sakuraの「記録庫」として利用します。

GitHubには、プログラムだけではなく、ドキュメントや設計資料なども保存します。

例えば、

- Pythonプログラム
- Markdown原稿
- README
- CHANGELOG
- 設計資料
- 画像
- その他のプロジェクト関連ファイル

などを管理します。

GitHubを利用する大きな理由の一つが、変更履歴を残せることです。

例えば、

Version 0.1 ↓ Version 0.2 ↓ Version 0.3

というように、Project Sakuraがどのように変化してきたのかを確認できます。

間違ってファイルを変更した場合でも、以前の変更内容を確認できます。

Project

Sakuraでは、GitHubを単なる「ファイル置き場」としてではなく、開発記録を残す場所として利用します。

2.7 GitHub Desktopの役割

GitHub Desktopは、GitHubを扱いやすくするためのツールです。

Gitにはコマンドを入力して操作する方法があります。

しかし、Project

Sakuraでは、最初からすべてのGitコマンドを覚える必要はありません。

GitHub Desktopを使えば、

変更されたファイルを確認する ↓ 変更内容を確認する ↓ Commitする ↓

GitHubへ送信する ↓ 履歴を確認する

という作業を画面上で行うことができます。

初心者の段階では、まずGitHub Desktopを利用して、

「ファイルを変更したらCommitする」

という基本的な習慣を身につけます。

2.8 VS Codeの役割

Visual Studio Code、通称VS Codeは、Project Sakuraの主要な編集環境です。

VS Codeでは、

- Python
- Markdown
- その他の設定ファイル

などを編集します。

Project Sakuraでは、プログラムだけではなく、本の原稿もVS Codeで管理します。

例えば第1章は、

```
``` text
docs/Vol01_基礎編/source/Chapter01.md
```
```

として保存しています。

第2章は、

```
``` text
docs/Vol01_基礎編/source/Chapter02.md
```
```

として保存します。

このように、各章を1つのMarkdownファイルとして管理します。

これにより、章ごとの内容を分かりやすく管理できます。

2.9 WordPressの役割

WordPressは、完成した記事をWeb上へ公開するための中心的な場所として利用します。

Project

Sakuraでは、WordPressを単なるブログ画面としてではなく、記事を公開・管理するためのWebシステムとして考えます。

基本的な流れは、

記事を企画する ↓ 情報を調査する ↓ 記事を作成する ↓ 内容を確認する ↓
画像を準備する ↓ WordPressへ登録する ↓ 公開する ↓ 結果を確認する

となります。

将来的には、Pythonなどを利用して、WordPressとの連携も検討します。

ただし、最初から自動投稿を行うものではありません。

まずは手作業で仕組みを理解し、その後に自動化できる部分を探します。

2.10 データ管理の役割

Project Sakuraでは、データを蓄積することも重要です。

例えば、

- 記事タイトル
- URL
- 投稿日
- カテゴリ
- キーワード
- 記事作成状況
- 更新日
- アクセス状況
- 改善内容

などを記録できます。

最初はExcelやCSV、Markdownなど、扱いやすい形式から始めても構いません。

重要なのは、

「作ったものを記録する」

ことです。

記録が残っていれば、

どの記事を作ったのか ↓ どの記事が読まれたのか ↓ どの記事を改善したのか ↓

次に何を作るのか

という判断につながられます。

2.11 画像作成の役割

ブログ記事では、文章だけではなく画像も重要になります。

Project Sakuraでは、記事の内容に合わせて、

- アイキャッチ画像
- 説明用画像
- 図解
- その他のブログ用素材

などを準備します。

画像についても、記事作成と同じように、

企画 ↓ 作成 ↓ 確認 ↓ 保存 ↓ 公開

という流れで管理します。

画像ファイルについても、必要に応じてGitHubの管理対象として整理します。

2.12 Project Sakuraの基本ワークフロー

Project Sakuraでは、ブログ運営を次のような流れで考えます。

`` ` text

企画

↓

調査

↓

AIによる整理・支援

↓

記事作成

↓

内容確認

↓

画像作成

↓

WordPressへ登録

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の企画

...

この流れを繰り返すことがProject Sakuraの基本です。

重要なのは、記事を1本公開して終わりにしないことです。

公開後の結果を確認し、その結果を次の記事や既存記事の改善に利用します。

2.13 「作る」だけではなく「改善する」

Project Sakuraでは、記事を作ることだけを成功としません。

例えば、記事を公開しても、ほとんど読まれない場合があります。

その場合、

タイトルを変更する ↓ 見出しを改善する ↓ 内容を追加する ↓ 画像を改善する

↓ 内部リンクを見直す ↓ 再度結果を確認する

という改善を行います。

つまり、

作る ↓ 公開する ↓ 確認する ↓ 改善する

というサイクルを回します。

この考え方は、Project Sakura全体に共通します。

2.14 手作業から自動化へ

Project Sakuraでは、最初から完全自動化を目指しません。

まずは手作業で実際の流れを経験します。

例えば記事作成なら、

段階1 手作業

人がすべての作業を行います。

段階2 手順化

作業の順番を決めます。

段階3 半自動化

一部の作業をPythonなどで補助します。

段階4 自動化

繰り返し作業を可能な範囲で自動化します。

この順番を大切にします。

手順を理解しないまま自動化すると、問題が発生したときに原因が分からなくなる可能性があります。

そのため、

「まず自分でやってみる」

ことを基本とします。

2.15 人が担当する部分

Project Sakuraでは、人の判断を重要視します。

例えば、

- ブログの方向性を決める
- 読者を考える
- 記事テーマを決める
- 情報の正確性を確認する
- 公開するか判断する
- 改善方針を決める

といった部分です。

これらは、単純に自動化すればよいものではありません。

Project

Sakuraでは、人が考える部分と、コンピューターに任せる部分を整理します。

2.16 コンピューターに任せる部分

一方で、繰り返し作業には自動化できるものがあります。

例えば、

- ファイル名を整理する
- データを並べ替える
- CSVを読み込む
- 定型的な文章を生成する
- データを集計する
- ファイルをコピーする
- 定型処理を繰り返す

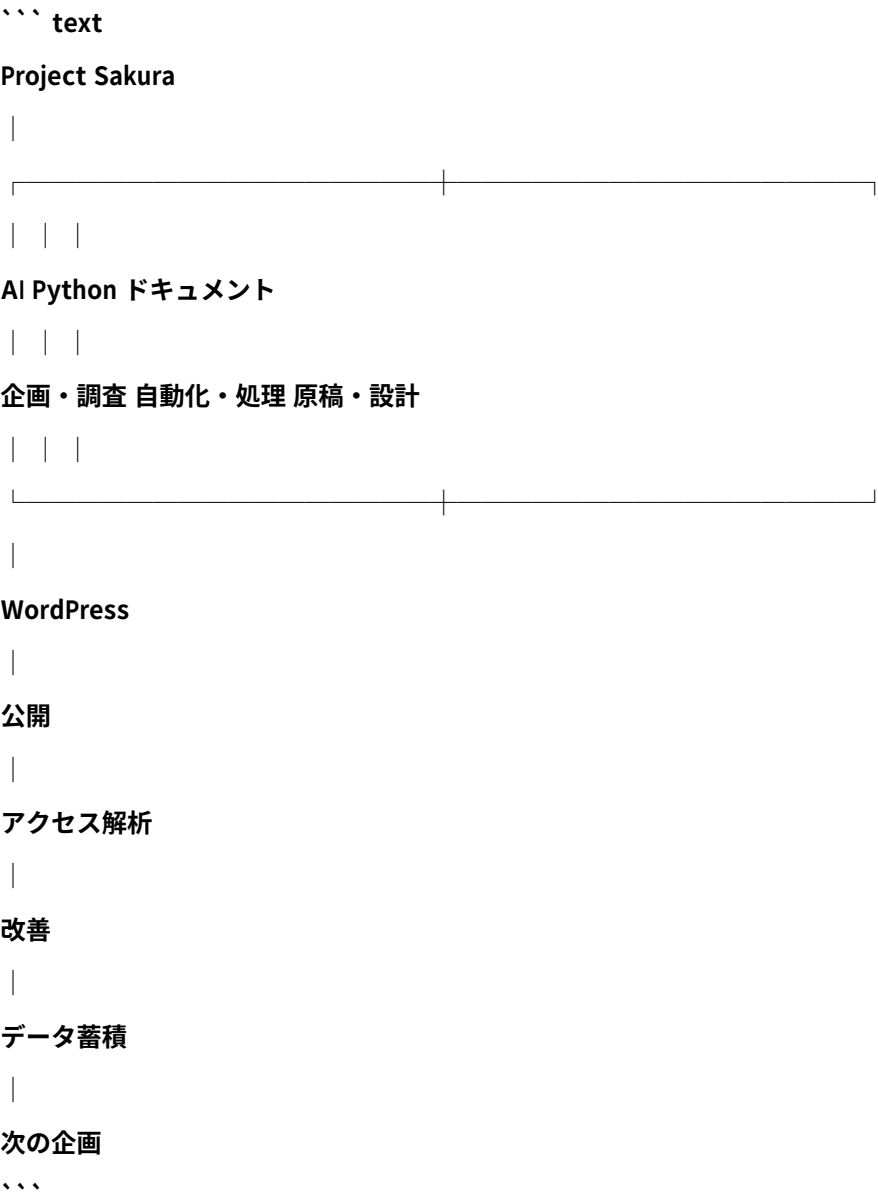
などです。

このような作業はPythonなどを使って自動化できる可能性があります。

ただし、実際に自動化するかどうかは、作業量や安全性を確認した上で判断します。

2.17 Project Sakuraの全体構造

Project Sakuraの全体構造を簡単に表すと、次のようになります。

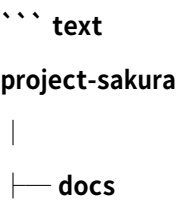


この構造を基本として、今後少しずつ具体的な機能を追加していきます。

2.18 Project Sakuraのドキュメント構成

Project Sakuraでは、本書そのものもプロジェクトの一部として管理します。

現在の基本構成は次のとおりです。




```
| |
| |— CHANGELOG.md
| |
| |— Vol01_基礎編
| |
| |— images
| |
| |— source
| | |— Chapter01.md
| | |— Chapter02.md
| |
| |— Project_Sakura_Vol1.docx
| |— Project_Sakura_Vol1.pdf
| |— README.md
|
|— python
| |— lesson01.py
|
|— .gitattributes
|
|— README.md
...

```

第1章、第2章というように、各章をそれぞれ1つのMarkdownファイルとして管理します。

これにより、原稿を修正した場合でも、どの章が変更されたのか分かりやすくなります。

2.19 本書の制作方法

Project

Sakuraでは、本書も通常のプログラム開発と同じように、段階的に作成します。

基本的な流れは、

原稿を作る ↓ 内容を確認する ↓ 修正する ↓ 章を完成させる ↓

GitHubへ保存する ↓ 正式版のWord・PDFへ反映する

という流れです。

各章を個別に管理することで、後から内容を修正しやすくします。

また、正式版のWord・PDFは、確定した原稿から作成します。

2.20 バージョン管理

Project Sakuraでは、文章もプログラムと同じようにバージョンを管理します。

例えば、

``` text

v0.1

↓

v0.2

↓

v0.3

↓

v1.0

```

というように、内容が成長していきます。

最初の段階では、完璧な本を作る必要はありません。

まずはv0.1として完成させます。

その後、内容を追加したり、説明を改善したりしながら、次のバージョンへ進めます。

2.21 エラーや失敗も資産にする

Project Sakuraでは、失敗も記録します。

例えば、

- ファイルの保存場所を間違えた
- Pythonが実行できなかった
- GitHubへの保存方法が分からなかった
- PDFを作成したが表示できなかった
- Markdownの書式が崩れた

といった問題が発生することがあります。

これらは無駄ではありません。

問題が発生したときに、

何が起きたのか ↓ 原因は何だったのか ↓ どうやって解決したのか ↓
次回どうすればよいのか

を記録しておけば、今後の作業に役立ちます。

Project Sakuraでは、こうした経験も開発記録として残します。

2.22 初心者でも進められる設計

Project Sakuraでは、初心者でも作業を進められることを重視します。

そのため、難しい機能を一気に作るのではなく、

小さな作業 ↓ 動作確認 ↓ 理解 ↓ 次の作業

という流れを繰り返します。

例えばPythonなら、

文字を表示する ↓ 変数を使う ↓ 条件分岐を使う ↓ 繰り返し処理を使う ↓
ファイルを扱う ↓ データを扱う ↓ Webと連携する

というように段階的に進めます。

分からない部分があれば、その部分まで戻って確認します。

2.23 Project Sakuraの成長イメージ

Project Sakuraは、最初は非常に小さなプロジェクトです。

最初に作ったPythonプログラムも、画面に文字を表示するだけでした。

しかし、

小さなプログラム ↓ 小さな自動化 ↓ 複数の処理 ↓ ブログ運営支援 ↓
データ管理 ↓ 運営システム

というように、少しずつ大きくしていきます。

重要なのは、最初から大きなものを作ろうとしないことです。

一つの小さな機能を完成させ、それを積み重ねます。

2.24 第2章まとめ

Project Sakuraは、

AI + Python + GitHub + VS Code + WordPress + データ管理

などを組み合わせて、ブログ運営を継続的に改善する仕組みを作るプロジェクトです。

それぞれの役割を整理すると、

AI → 企画・調査・文章作成などを支援する

Python → 繰り返し作業やデータ処理を自動化する

GitHub → プログラムや原稿、変更履歴を管理する

VS Code → プログラムやMarkdown原稿を編集する

WordPress → 記事をWeb上へ公開する

データ管理 → 作ったものや結果を記録する

という関係になります。

そして、これらを一つの流れとしてつなげます。

```
``` text
```

企画

↓

調査

↓

AI支援

↓

記事作成

↓

画像作成

↓

公開

↓

アクセス確認

↓

改善

↓

データ蓄積

↓

次の企画

...

Project Sakuraでは、この循環を少しずつ改善していきます。

最初は手作業でも構いません。

作業を理解し、手順を整理し、その後で自動化します。

この考え方を基本として、次の章から具体的な設計へ進みます。

-----

## Chapter Check

第2章を読み終えたら、次の質問を確認してみましょう。

### Q1

Project

Sakuraでは、なぜブログを作るだけでなく、運営の仕組みまで作ろうとしているのでしょうか？

### Q2

AIはProject Sakuraでどのような役割を担当しますか？

### Q3

PythonはProject Sakuraで何を自動化するために利用しますか？

### Q4

GitHubを利用する主な理由は何ですか？

### Q5

VS Codeは何を編集するために利用しますか？

### Q6

WordPressはProject Sakuraの中でどのような役割を担当しますか？

### Q7

なぜ最初から完全自動化を目指さないのでしょうか？

### Q8

Project Sakuraの基本的な運営サイクルを説明してください。

-----

## 第3章への予告

第2章では、Project

Sakuraを構成する主要な要素と、それぞれの役割を整理しました。

次の章では、さらに具体的な設計へ進みます。

ブログの目的、読者、ブログのテーマ、カテゴリ、記事構成、キーワード、コンテンツ管理、収益化、運営方法などについて整理します。

Project

Sakuraという「仕組み」を実際に動かすためには、まず「何を発信するブログなのか」を明確にする必要があります。

第3章では、

**\*\*「ブログ設計とコンテンツ戦略」\*\***

をテーマとして、Project Sakuraのブログ部分を具体的に設計していきます。